

KUPANGA — SPÉCIFICATIONS TECHNIQUES FRONT-END

Sommaire

1. [Configuration & Environnement](#)
2. [Architecture Générale](#)
3. [Modèles TypeScript](#)
4. [Services & Intercepteurs](#)
5. [Guards](#)
6. [Routage](#)
7. [Module Biens](#)
8. [Module Contrats](#)
9. [Module États des Lieux](#)
10. [Module Quittances](#)
11. [Dashboard](#)
12. [Module Chat](#)
13. [Pages Publiques de Signature](#)
14. [Récapitulatif de tous les appels API](#)

1. Configuration & Environnement

Fichiers d'environnement

`src/environments/environment.ts` (développement)

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8089',  
  wsUrl: 'http://localhost:8089/ws'  
};
```

`src/environments/environment.prod.ts` (production)

```
export const environment = {  
  production: true,  
  apiUrl: 'https://kupanga-api.onrender.com',  
  wsUrl: 'https://kupanga-api.onrender.com/ws'  
};
```

Configuration applicative — `app.config.ts`

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideBrowserGlobalErrorListeners(),  
    provideRouter(routes, withComponentInputBinding()),  
    provideHttpClient(  
      withFetch(),  
      withInterceptors([jwtInterceptor, errorInterceptor])  
    )  
  ]  
};
```

2. Architecture Générale

```
src/app/  
├─ core/  
│   ├─ guards/      auth.guard.ts | no-auth.guard.ts | role.guard.ts  
│   └─ interceptors/ jwt.interceptor.ts | error.interceptor.ts
```

```

|   |   | models/      auth.models.ts | user.models.ts | bien.models.ts |
|   |   | |           | contrat.models.ts | edl.models.ts | quittance.models.ts | chat.models.ts
|   |   | |           | services/      auth.service.ts | bien.service.ts | contrat.service.ts |
|   |   | |           | |           | edl.service.ts | quittance.service.ts | chat.service.ts | toast.service.ts
|   |   | |           |
|   |   | | features/
|   |   | | | biens/      biens-liste | bien-detail | bien-form | mes-biens | mon-logement
|   |   | | | contrats/  contrat-form | contrats-liste
|   |   | | | etats-des-lieux/ edl-form | edl-liste
|   |   | | | quittances/ quittance-form | quittances-liste
|   |   | | | dashboard/  proprietaire-dashboard | locataire-dashboard
|   |   | | | chat/      chat
|   |   | | | public/
|   |   | | | | signature-contrat/
|   |   | | | | signature-edl/
|   |   | | | landing/
|   |   | | | auth/
|   |   | | layouts/
|   |   | | | shared/

```

Principes architecturaux :

- Tous les composants sont **standalone** (imports explicites dans le décorateur)
- Gestion d'état via **Angular Signals** (`signal()`, `computed()`)
- **Reactive Forms** (FormBuilder + Validators)
- **Intercepteurs HTTP** pour JWT et gestion d'erreurs
- Toutes les listes sont **paginées côté serveur**

3. Modèles TypeScript

3.1 Modèles User

```

// core/models/user.models.ts

export type Role = 'ROLE_PROPRIETAIRE' | 'ROLE_LOCATAIRE' | 'ROLE_ADMIN';

export interface UserDTO {
  id: number;
  firstName: string;
  lastName: string;
  mail: string;
  role: Role;
  urlProfile?: string;
  hasCompleteProfil: boolean;
}

export interface LocataireSearchDTO {
  nom?: string;
  prenom?: string;
  email?: string;
  page?: number;
  size?: number;
}

export interface LocatairePageDTO {
  contenu: UserDTO[];
  pageActuelle: number;
  totalPages: number;
  totalElements: number;
  dernierePage: boolean;
  premierePage: boolean;
}

```

3.2 Modèles Bien

```

// core/models/bien.models.ts

export type TypeBien = 'APPARTEMENT' | 'MAISON' | 'STUDIO';

```

```

export type ModeChauffage =
  | 'ELECTRIQUE' | 'GAZ' | 'FIOUL' | 'BOIS'
  | 'POMPE_A_CHALEUR' | 'POELE' | 'COLLECTIF' | 'SANS_CHAUFFAGE';

export type ClasseEnergie = 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G';
export type ClasseGes      = 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G';

export type PoiType = 'SCHOOL' | 'HOSPITAL' | 'PHARMACY' | 'KINDERGARTEN';

export interface BienDTO {
  id: number;
  titre: string;
  typeBien: TypeBien;
  description: string;
  adresse: string;
  ville: string;
  codePostal: string;
  pays: string;
  latitude?: number;
  longitude?: number;
  surfaceHabitable: number;
  nombrePieces: number;
  nombreChambres: number;
  etage?: number;
  ascenseur: boolean;
  anneeConstruction?: number;
  modeChauffage: ModeChauffage;
  classeEnergie: ClasseEnergie;
  classeGes: ClasseGes;
  loyerMensuel: number;
  chargesMensuelles: number;
  depotGarantie: number;
  meuble: boolean;
  colocation: boolean;
  disponibleDe?: string;           // ISO date
  proprietaire: UserDTO;
  locataire?: UserDTO;
  images: string[];               // URLs
  documents: string[];           // URLs
  pois: PoiType[];
  createdAt: string;
  updatedAt: string;
}

export interface BienFormDTO {
  titre: string;
  typeBien: TypeBien;
  description: string;
  adresse: string;
  ville: string;
  codePostal: string;
  pays: string;
  surfaceHabitable: number;
  nombrePieces: number;
  nombreChambres: number;
  etage?: number;
  ascenseur: boolean;
  anneeConstruction?: number;
  modeChauffage: ModeChauffage;
  classeEnergie: ClasseEnergie;
  classeGes: ClasseGes;
  loyerMensuel: number;
  chargesMensuelles: number;
  depotGarantie: number;
  meuble: boolean;
  colocation: boolean;
  disponibleDe?: string;
}

export interface BienSearchDTO {
  villes?: string[];
  typesBien?: TypeBien[];
}

```

```

titre?: string;
loyerMin?: number;
loyerMax?: number;
surfaceMin?: number;
surfaceMax?: number;
piecesMin?: number;
ascenseur?: boolean;
meuble?: boolean;
colocation?: boolean;
disponibleAvant?: string;
classesEnergie?: ClasseEnergie[];
classesGes?: ClasseGes[];
modesChauffage?: ModeChauffage[];
poisRequis?: PoiType[];
page?: number;
size?: number;
sortBy?: string;
sortDirection?: 'ASC' | 'DESC';
}

export interface BienPageDTO {
  contenu: BienDTO[];
  pageActuelle: number;
  totalPages: number;
  totalElements: number;
  dernierePage: boolean;
  premierePage: boolean;
}

```

3.3 Modèles Contrat

```

// core/models/contrat.models.ts

export type StatutContrat =
  | 'BROUILLON'
  | 'EN_ATTENTE_SIGNATURE_PROPRIO'
  | 'EN_ATTENTE_SIGNATURE_LOCATAIRE'
  | 'SIGNE'
  | 'EXPIRE'
  | 'ANNULE';

export interface ContratDTO {
  id: number;
  bienId: number;
  adresseBien: string;
  proprietaire: UserDTO;
  locataire: UserDTO;
  loyerMensuel: number;
  chargesMensuelles: number;
  depotGarantie: number;
  dateDebut: string;
  dateFin?: string;
  dureeBailMois: number;
  proprietaireASigné: boolean;
  locataireASigné: boolean;
  urlPdf?: string;
  statut: StatutContrat;
  dateSignatureProprietaire?: string;
  dateSignatureLocataire?: string;
  createdAt: string;
  updatedAt: string;
}

export interface ContratFormDTO {
  bienId: number;
  emailLocataire: string;
  dateDebut: string;
  dateFin?: string;
  dureeBailMois: number;
  loyerMensuel: number;
  chargesMensuelles: number;
}

```

```

    depotGarantie: number;
  }

export interface ContratSearchDTO {
  statut?: StatutContrat;
  loyerMin?: number;
  loyerMax?: number;
  dateDebutAfter?: string;
  dateDebutBefore?: string;
  page?: number;
  size?: number;
  sortBy?: string;
  sortDirection?: 'ASC' | 'DESC';
}

export interface SignatureDTO {
  signatureBase64: string; // Image PNG en base64 depuis canvas
}

```

3.4 Modèles État des Lieux

```

// core/models/edl.models.ts

export type TypeEtat = 'ENTREE' | 'SORTIE';
export type StatutEdl = 'BROUILLON' | 'EN_ATTENTE_SIGNATURE_PROPRIO' | 'EN_ATTENTE_SIGNATURE_LOCATAIRE' |
'SIGNE' | 'EXPIRE';
export type TypeElement = 'MUR' | 'PLAFOND' | 'SOL' | 'FENETRE' | 'PORTE' | 'VOLET' | 'PRISE' | 'LUMINAIRE' |
'RADIATEUR' | 'EQUIPEMENT' | 'AUTRE';
export type EtatElement = 'BON' | 'USAGE_NORMAL' | 'MAUVAIS' | 'HORS_SERVICE';
export type TypeCompteur = 'EAU_FROIDE' | 'EAU_CHAUDE' | 'ELECTRICITE_HP' | 'ELECTRICITE_HC' | 'GAZ';

export interface ElementDTO {
  type: TypeElement;
  etat: EtatElement;
  commentaire?: string;
}

export interface PieceDTO {
  nom: string;
  elements: ElementDTO[];
}

export interface CompteurDTO {
  type: TypeCompteur;
  valeur: number;
  unite: string;
}

export interface CleDTO {
  description: string;
  quantite: number;
}

export interface EtatDesLieuxFormDTO {
  bienId: number;
  emailLocataire: string;
  type: TypeEtat;
  dateRealisation: string;
  heureRealisation?: string;
  observations?: string;
  compteurs: CompteurDTO[];
  cles: CleDTO[];
  pieces: PieceDTO[];
}

export interface EtatDesLieuxDTO {
  id: number;
  type: TypeEtat;
  statut: StatutEdl;
  dateRealisation: string;
  heureRealisation?: string;
}

```

```

observations?: string;
urlPdf?: string;
dateSignatureProprietaire?: string;
dateSignatureLocataire?: string;
nomProprietaire: string;
emailProprietaire: string;
nomLocataire: string;
emailLocataire: string;
adresseBien: string;
typeBien: TypeBien;
pieces: PieceDTO[];
compteurs: CompteurDTO[];
cles: CleDTO[];
}

export interface EdlSearchDTO {
  bienId?: number;
  type?: TypeEtat;
  statut?: StatutEdl;
  page?: number;
  size?: number;
  sortBy?: string;
  sortDirection?: 'ASC' | 'DESC';
}

export interface EdlPageDTO {
  contenu: EtatDeslieuxDTO[];
  pageActuelle: number;
  totalPages: number;
  totalElements: number;
  dernierePage: boolean;
  premierePage: boolean;
}

```

3.5 Modèles Quittance

```

// core/models/quittance.models.ts

export type StatutQuittance = 'EN_ATTENTE' | 'PAYEE' | 'EN_RETARD' | 'IMPAYEE';

export interface QuittanceDTO {
  id: number;
  mois: number;
  annee: number;
  moisLabel: string; // ex: "Janvier 2025"
  loyerMensuel: number;
  chargesMensuelles: number;
  montantTotal: number;
  dateEcheance: string;
  datePaieement?: string;
  statut: StatutQuittance;
  urlPdf?: string;
  nomProprietaire: string;
  emailProprietaire: string;
  nomLocataire: string;
  emailLocataire: string;
  adresseBien: string;
  typeBien: string;
  surfaceHabitable: number;
  createdAt: string;
  updatedAt: string;
}

export interface QuittanceFormDTO {
  bienId: number;
  emailLocataire: string;
  contratId?: number;
  mois: number; // 1-12
  annee: number;
  loyerMensuel?: number;
  chargesMensuelles?: number;
}

```

```
    dateEcheance: string;
  }

export interface QuittanceSearchDTO {
  annee?: number;
  mois?: number;
  statut?: StatutQuittance;
  bienId?: number;
  page?: number;
  size?: number;
  sortBy?: string;
  sortDirection?: 'ASC' | 'DESC';
}

export interface QuittancePageDTO {
  contenu: QuittanceDTO[];
  pageActuelle: number;
  totalPages: number;
  totalElements: number;
  dernierePage: boolean;
  premierePage: boolean;
}
```

3.6 Modèles Chat

```
// core/models/chat.models.ts

export interface ConversationDTO {
  id: number;
  bienId: number;
  bienTitre: string;
  emailExpéditeur: string;
  emailDestinataire: string;
  lastMessage?: string;
  lastMessageAt?: string;
  createdAt: string;
  nonLuCount: number;
}

export interface ConversationPageDTO {
  contenu: ConversationDTO[];
  pageActuelle: number;
  totalPages: number;
  totalElements: number;
  dernierePage: boolean;
  premierePage: boolean;
}

export interface ConversationSearchDTO {
  nomDuBien?: string;
  lu?: boolean;
  page?: number;
  size?: number;
  sortBy?: string;
  sortDirection?: 'ASC' | 'DESC';
}

export interface MessageDTO {
  id: number;
  contenu: string;
  lu: boolean;
  createdAt: string;
  expéditeurId: number;
  expéditeurNom: string;
  expéditeurEmail: string;
  destinataireId: number;
  destinataireNom: string;
  destinataireEmail: string;
  bienId: number;
  bienAdresse?: string;
  conversationId?: number;
```

```

}

export interface NotificationDTO {
  conversationId: number;
  expéditeurEmail: string;
  expéditeurNom: string;
  contenuPreview: string;
  createdAt: string;
}

// Modèle UI interne (non-API)
export interface ChatMessage {
  type: 'message' | 'separator';
  // Si type === 'message'
  id?: number;
  contenu?: string;
  direction?: 'sent' | 'received';
  time?: string;
  isNew?: boolean;
  // Si type === 'separator'
  label?: string;
}

```

4. Services & Intercepteurs

4.1 JwtInterceptor

Fichier : `core/interceptors/jwt.interceptor.ts`

Comportement :

1. Récupère le token depuis `AuthService`
2. Si token présent ET la requête n'est pas une route publique → ajoute le header `Authorization: Bearer {token}`
3. Sur erreur 401 :
 - Si pas déjà une requête retry : appelle `POST /auth/refresh`
 - Si refresh OK → relance la requête originale avec le nouveau token
 - Si refresh KO → efface la session + redirige vers `/login`

Routes exclues de l'injection de token :

- `/auth/login, /auth/register, /auth/refresh`
- `/contrats/signer/` (routes publiques de signature)
- `/etats-des-lieux/signer/` (routes publiques de signature)

```

// Logique de retry anti-boucle infinie
const IS_RETRY = new HttpContextToken<boolean>(() => false);

// Dans l'intercepteur
if (err.status === 401 && !req.context.get(IS_RETRY)) {
  return authService.refreshToken().pipe(
    switchMap(newToken => {
      const retryReq = req.clone({
        setHeaders: { Authorization: `Bearer ${newToken}` },
        context: new HttpContext().set(IS_RETRY, true)
      });
      return next(retryReq);
    }),
    catchError(() => {
      authService.clearSession();
      router.navigate(['/login']);
      return throwError(() => err);
    })
  );
}

```

4.2 ErrorInterceptor

Fichier : `core/interceptors/error.interceptor.ts`

Comportement :

- Intercepte toutes les erreurs HTTP
- Affiche un toast via `ToastService`
- `status === 0` → toast "Impossible de joindre le serveur..."
- `status >= 500` → toast "Erreur serveur..."
- Autres → extrait le message de `error.message` ou `error.error`
- Propage l'erreur enrichie avec une propriété `userMessage`

4.3 BienService

Fichier : `core/services/bien.service.ts`

```
// Injection : inject(HttpClient) + inject(environment)
// Base URL : environment.apiUrl

search(dto: BienSearchDTO): Observable<BienPageDTO>
  → POST {apiUrl}/biens/search
  → Body : BienSearchDTO
  → Retour: BienPageDTO

getId(id: number): Observable<BienDTO>
  → GET {apiUrl}/biens/:id
  → Retour: BienDTO

create(formData: FormData): Observable<void>
  → POST {apiUrl}/biens
  → Body : FormData (champs + fichiers images)
  → Retour: void

getMesBiens(): Observable<BienDTO[]>
  → GET {apiUrl}/users/biens
  → Retour: BienDTO[]

assignLocataire(bienId: number, userId: number): Observable<void>
  → POST {apiUrl}/biens/:bienId/assigne-locataire/:userId
  → Retour: void

rechercheLocataire(bienId: number, dto: LocataireSearchDTO): Observable<LocatairePageDTO>
  → POST {apiUrl}/users/:bienId/recherche-locataire
  → Body : LocataireSearchDTO
  → Retour: LocatairePageDTO
```

4.4 ContratService

Fichier : `core/services/contrat.service.ts`

```
create(dto: ContratFormDTO): Observable<void>
  → POST {apiUrl}/contrats
  → Body : ContratFormDTO
  → Retour: void

signerProprietaire(id: number, dto: SignatureDTO): Observable<void>
  → POST {apiUrl}/contrats/:id/signer-proprio
  → Body : SignatureDTO { signatureBase64: string }
  → Retour: void

getToken(token: string): Observable<ContratDTO>
  → GET {apiUrl}/contrats/signer/:token
  → Sans JWT (public)
  → Retour: ContratDTO

signerParToken(token: string, dto: SignatureDTO): Observable<void>
  → POST {apiUrl}/contrats/signer/:token
  → Body : SignatureDTO { signatureBase64: string }
  → Sans JWT (public)
  → Retour: void

search(dto: ContratSearchDTO): Observable<ContratPageDTO>
```

```
→ POST {apiUrl}/contrats/search
→ Body : ContratSearchDTO
→ Retour: ContratPageDTO
```

4.5 EdlService

Fichier : `core/services/edl.service.ts`

```
create(dto: EtatDesLieuxFormDTO): Observable<void>
→ POST {apiUrl}/etats-des-lieux
→ Body : EtatDesLieuxFormDTO
→ Retour: void

signerProprietaire(id: number, dto: SignatureDTO): Observable<void>
→ POST {apiUrl}/etats-des-lieux/:id/signer-proprietaire
→ Body : SignatureDTO { signatureBase64: string }
→ Retour: void

getByToken(token: string): Observable<EtatDesLieuxDTO>
→ GET {apiUrl}/etats-des-lieux/signer/:token
→ Sans JWT (public)
→ Retour: EtatDesLieuxDTO

signerParToken(token: string, dto: SignatureDTO): Observable<void>
→ POST {apiUrl}/etats-des-lieux/signer/:token
→ Body : SignatureDTO { signatureBase64: string }
→ Sans JWT (public)
→ Retour: void

search(dto: EdlSearchDTO): Observable<EdlPageDTO>
→ POST {apiUrl}/etats-des-lieux/search
→ Body : EdlSearchDTO
→ Retour: EdlPageDTO
```

4.6 QuittanceService

Fichier : `core/services/quittance.service.ts`

```
create(dto: QuittanceFormDTO): Observable<void>
→ POST {apiUrl}/quittances
→ Body : QuittanceFormDTO
→ Retour: void

marquerPayee(id: number, dto: SignatureDTO): Observable<void>
→ POST {apiUrl}/quittances/:id/marquer-payee
→ Body : SignatureDTO { signatureBase64: string }
→ Retour: void

getByBien(bienId: number): Observable<QuittanceDTO[]>
→ GET {apiUrl}/quittances/bien/:bienId
→ Retour: QuittanceDTO[]

getMesQuittances(): Observable<QuittanceDTO[]>
→ GET {apiUrl}/quittances/mes-quittances
→ Retour: QuittanceDTO[]

getId(id: number): Observable<QuittanceDTO>
→ GET {apiUrl}/quittances/:id
→ Retour: QuittanceDTO

search(dto: QuittanceSearchDTO): Observable<QuittancePageDTO>
→ POST {apiUrl}/quittances/search
→ Body : QuittanceSearchDTO
→ Retour: QuittancePageDTO
```

4.7 ChatService

Fichier : `core/services/chat.service.ts`

WebSocket (STOMP over SockJS)

```
// Dépendances : @stomp/stompjs + sockjs-client
// Endpoint WS : environment.wsUrl ex: http://localhost:8089/ws

connect(userEmail: string): void
// Crée SockJS socket → Client STOMP
// S'abonne à 2 topics :
//   /user/{email}/queue/messages      → Subject<MessageDTO> messages$
//   /user/{email}/queue/notifications → Subject<NotificationDTO> notifications$
// Met à jour le signal wsStatus: 'connected' | 'connecting' | 'disconnected'

disconnect(): void
// Déconnecte le client STOMP

send(contenu: string, emailDestinataire: string, bienId: number): void
// Publie sur /app/chat.send
// Body JSON : { contenu, emailDestinataire, bienId }
```

API REST

```
searchConversations(dto: ConversationSearchDTO): Observable<ConversationPageDTO>
→ POST {apiUrl}/conversations/search
→ Body : ConversationSearchDTO
→ Retour: ConversationPageDTO

getHistorique(bienId: number, emailInterlocuteur: string): Observable<MessageDTO[]>
→ GET {apiUrl}/historique?bienId={bienId}&emailInterlocuteur={email}
→ Retour: MessageDTO[]

marquerLus(emailInterlocuteur: string): Observable<void>
→ POST {apiUrl}/messages/conversation/:emailInterlocuteur/lire
→ Retour: void

countNonLus(): Observable<number>
→ GET {apiUrl}/messages/non-lus
→ Retour: number
```

Signaux exposés

```
wsStatus      = signal<'connected' | 'connecting' | 'disconnected'>('disconnected')
unreadCount   = signal<number>(0)
unreadConvos  = signal<number[]>([]) // IDs de conversations non lues
messages$     = new Subject<MessageDTO>()
notifications$ = new Subject<NotificationDTO>()
```

4.8 ToastService

Fichier : `core/services/toast.service.ts`

```
interface Toast {
  id: string;
  message: string;
  type: 'success' | 'error' | 'info' | 'warning';
}

// Signal
toasts = signal<Toast[]>([])

success(message: string): void // Auto-suppression après 4s
error(message: string): void
info(message: string): void
```

```
warning(message: string): void
remove(id: string): void
```

5. Guards

5.1 AuthGuard

Fichier : `core/guards/auth.guard.ts`

```
// Attend que sessionInitialized (signal de AuthService) soit true
// Si non connecté → navigate(['/login'], queryParams: { redirect: currentUrl })
// Si connecté → retourne true
```

5.2 NoAuthGuard

Fichier : `core/guards/no-auth.guard.ts`

```
// Si connecté :
//   ROLE_PROPRIETAIRE → navigate('/proprietaire/biens')
//   ROLE_LOCATAIRE   → navigate('/locataire/mon-bien')
// Sinon → retourne true (accès à la page publique autorisé)
```

5.3 RoleGuard

Fichier : `core/guards/role.guard.ts`

```
// Fabrique de guard : roleGuard('ROLE_PROPRIETAIRE') ou roleGuard('ROLE_LOCATAIRE')
// Vérifie user.role === requiredRole
// Si rôle incorrect :
//   ROLE_PROPRIETAIRE → navigate('/proprietaire/dashboard')
//   ROLE_LOCATAIRE   → navigate('/locataire/dashboard')
```

6. Routage

```
// app.routes.ts

const routes: Routes = [

  // Routes publiques
  { path: '', component: LandingComponent, canActivate: [noAuthGuard] },
  { path: 'biens', component: BiensListeComponent },
  { path: 'biens/:id', component: BienDetailComponent },
  { path: 'login', component: LoginComponent, canActivate: [noAuthGuard] },
  { path: 'register', component: RegisterComponent, canActivate: [noAuthGuard] },
  { path: 'forgot-password', component: ForgotPasswordComponent, canActivate: [noAuthGuard] },
  { path: 'reset-password', component: ResetPasswordComponent, canActivate: [noAuthGuard] },

  // Signature publique (sans authentification)
  { path: 'contrats/signer/:token', component: SignatureContratComponent },
  { path: 'edl/signer/:token', component: SignatureEdlComponent },

  // Espace Propriétaire
  {
    path: 'proprietaire',
    canActivate: [AuthGuard, roleGuard('ROLE_PROPRIETAIRE')],
    component: ProprietaireLayoutComponent,
    children: [
      { path: 'dashboard', component: ProprietaireDashboardComponent },
      { path: 'biens/nouveau', component: BienFormComponent },
      { path: 'biens', component: MesBiensComponent },
      { path: 'annonces', component: BiensListeComponent },
    ]
  }
]
```

```

    { path: 'contrats/nouveau',      component: ContratFormComponent },
    { path: 'contrats',               component: ContratsListeComponent },
    { path: 'etats-des-lieux/nouveau', component: EdlFormComponent },
    { path: 'etats-des-lieux',        component: EdlListeComponent },
    { path: 'quittances/nouveau',    component: QuittanceFormComponent },
    { path: 'quittances',             component: QuittancesListeComponent },
    { path: 'chat',                   component: ChatComponent },
    { path: 'profil',                 component: ProfilComponent },
  ]
},

// Espace Locataire
{
  path: 'locataire',
  canActivate: [authGuard, roleGuard('ROLE_LOCATAIRE')],
  component: LocataireLayoutComponent,
  children: [
    { path: 'dashboard',      component: LocataireDashboardComponent },
    { path: 'mon-bien',       component: MonLogementComponent },
    { path: 'annonces',       component: BiensListeComponent },
    { path: 'contrats',       component: ContratsListeComponent },
    { path: 'etats-des-lieux', component: EdlListeComponent },
    { path: 'quittances',     component: QuittancesListeComponent },
    { path: 'chat',           component: ChatComponent },
    { path: 'profil',         component: ProfilComponent },
  ]
},

{ path: '**', redirectTo: '' }
];

```

7. Module Biens

7.1 BiensListeComponent

Route : `/biens` | `/proprietaire/annonces` | `/locataire/annonces`

Signaux d'état :

```

loading      = signal<boolean>(false)
biens        = signal<BienDTO[]>([])
currentPage  = signal<number>(0)
totalPages   = signal<number>(0)
totalElements = signal<number>(0)

```

Formulaire de recherche (FormBuilder) :

```

searchForm = fb.group({
  titre:      [''],
  villes:     [''],      // CSV ou multi-select
  typesBien:  [[]],      // TypeBien[]
  loyerMax:   [null],
  surfaceMin: [null],
  piecesMin:  [null],
  ascenseur:  [null],
  meuble:     [null],
  colocation: [null],
  poisRequis: [[]]
})
sortBy       = signal<string>('createdAt')
sortDirection = signal<'ASC'|'DESC'>('DESC')
pageSize     = 12

```

Logique :

1. Au `ngOnInit` → appelle `search(page=0)`

2. `search(page)` :
 - Construit `BienSearchDTO` depuis le formulaire
 - Appelle `BienService.search(dto)` → `POST /biens/search`
 - Met à jour `biens`, `currentPage`, `totalPages`
3. Pagination : boutons précédent/suivant appellent `search(page ± 1)`
4. Bouton "Contacter" :
 - Si utilisateur connecté → navigue vers `/proprietaire/chat` ou `/locataire/chat`
 - Sinon → navigue vers `/login?redirect=/biens/:id`
5. Clic sur une annonce → navigue vers `/biens/:id`

Affichage :

- Icône de type bien (APPARTEMENT / MAISON / STUDIO)
- Badge classe énergie (couleur selon A→G)
- Prix loyer + charges
- Ville + code postal
- Initiales du propriétaire en avatar
- Badges POI (icônes école, hôpital, pharmacie, crèche)

7.2 BienDetailComponent

Route : `/biens/:id`

Signaux d'état :

```

bien          = signal<BienDTO | null>(null)
loading       = signal<boolean>(false)
currentImageIdx = signal<number>(0)
chatOpen      = signal<boolean>(false)
chatMessages  = signal<ChatMessage[]>([])
chatInput     = signal<string>('')
chatLoading   = signal<boolean>(false)

```

Logique :

1. Récupère `id` depuis les paramètres de route (via `ActivatedRoute` ou `input()`)
2. Appelle `BienService.getById(id)` → `GET /biens/:id`
3. **Galerie d'images** :
 - `prevImage()` / `nextImage()` décrémentent/incrémentent `currentImageIdx`
 - Modulo sur `bien.images.length`
4. **Chat intégré** (ouverture via bouton "Contacter") :
 - Vérifie que l'utilisateur connecté n'est pas le propriétaire du bien
 - Appelle `ChatService.getHistorique(bienId, proprietaire.mail)` → `GET /historique?bienId=&emailInterlocuteur=`
 - Transforme `MessageDTO[]` → `ChatMessage[]` avec séparateurs de dates
 - S'abonne à `ChatService.messages$` pour les messages temps réel
 - Déduplique par `id` pour éviter les doublons WebSocket
 - `sendMessage()` : appelle `ChatService.send(contenu, proprietaireEmail, bienId)`
5. Si utilisateur non connecté → bouton redirige vers `/login`

7.3 BienFormComponent

Route : `/proprietaire/biens/nouveau`

Formulaire (FormBuilder + Validators) :

```

bienForm = fb.group({
  titre:          ['', [Validators.required, Validators.minLength(5), Validators.maxLength(100)]],
  typeBien:       ['APPARTEMENT', Validators.required],
  description:    ['', Validators.required],
  adresse:       ['', Validators.required],
  ville:         ['', Validators.required],
  codePostal:    ['', Validators.required],
  pays:          ['France', Validators.required],
  surfaceHabitable: [null, [Validators.required, Validators.min(5)]],

```

```

nombrePieces:      [null, [Validators.required, Validators.min(1)]],
nombreChambres:    [null, [Validators.required, Validators.min(0)]],
etage:             [null],
ascenseur:         [false],
anneeConstruction: [null],
modeChauffage:     ['ELECTRIQUE', Validators.required],
classeEnergie:     ['D', Validators.required],
classeGes:         ['D', Validators.required],
loyerMensuel:      [null, [Validators.required, Validators.min(1)]],
chargesMensuelles: [null, [Validators.required, Validators.min(0)]],
depotGarantie:     [null, [Validators.required, Validators.min(0)]],
meuble:           [false],
colocation:        [false],
disponibleDe:      [null]
})
images             = signal<File[]>([])
imagePreviews      = signal<string[]>([])
loading            = signal<boolean>(false)
error              = signal<string | null>(null)

```

Logique de soumission :

```

onSubmit(): void {
  // 1. Construit un FormData
  const fd = new FormData();
  // 2. Ajoute chaque champ du formulaire
  Object.entries(form.value).forEach(([k, v]) => fd.append(k, String(v)));
  // 3. Ajoute les fichiers images sous la clé 'images'
  images().forEach(file => fd.append('images', file));
  // 4. POST /biens
  bienService.create(fd).subscribe({
    next: () => router.navigate(['/proprietaire/biens']),
    error: (e) => error.set(e.userMessage)
  });
}

```

Upload d'images :

- Input file `accept="image/*"` multiple
- À chaque sélection → lit via `FileReader.readAsDataURL()` → stocke dans `imagePreviews`
- Affiche les previews sous le formulaire

7.4 MesBiensComponent

Route : `/proprietaire/biens`

Signaux d'état :

```

biens              = signal<BienDTO[]>([])
loading            = signal<boolean>(false)
showAssignDialog   = signal<boolean>(false)
selectedBienId     = signal<number | null>(null)
locataires         = signal<UserDTO[]>([])
locataireSearch    = signal<string>('')
locataireLoading   = signal<boolean>(false)
locatairePage      = signal<number>(0)
locataireTotalPages = signal<number>(0)

```

Logique :

1. Au `ngOnInit` → `BienService.getMesBiens()` → GET `/users/biens`
2. **Dialog d'assignation de locataire :**
 - Ouverture : `showAssignDialog.set(true)` + `selectedBienId.set(id)`
 - Recherche locataire : `BienService.rechercheLocataire(bienId, searchDTO)` → POST `/users/:bienId/recherche-locataire`
 - Champs de recherche : `nom`, `prenom`, `email` — 10 par page
 - Confirmation : `BienService.assignLocataire(bienId, userId)` → POST `/biens/:bienId/assigne-locataire/:userId`

- Après assignation → recharge la liste

7.5 MonLogementComponent

Route : `/locataire/mon-bien`

Signaux d'état :

```
bien          = signal<BienDTO | null>(null)
loading       = signal<boolean>(false)
currentImage  = signal<number>(0)
```

Logique :

1. Appelle `BienService.getMesBiens()` → GET `/users/biens`
2. Prend le premier élément du tableau (le locataire n'a qu'un seul logement)
3. Galerie : `prevImage()` / `nextImage()` avec modulo
4. Bouton "Contacter le propriétaire" → navigue vers `/locataire/chat`

8. Module Contrats

8.1 ContratFormComponent

Route : `/proprietaire/contrats/nouveau`

Signaux d'état :

```
biens          = signal<BienDTO[]>([])
biensLoading   = signal<boolean>(false)
loading        = signal<boolean>(false)
error          = signal<string | null>(null)
```

Formulaire :

```
contratForm = fb.group({
  bienId:      [null, Validators.required],
  emailLocataire: ['', [Validators.required, Validators.email]],
  dateDebut:   ['', Validators.required],
  dateFin:     [null],
  dureeBailMois: [12, [Validators.required, Validators.min(1)]],
  loyerMensuel: [null, [Validators.required, Validators.min(1)]],
  chargesMensuelles: [null, [Validators.required, Validators.min(0)]],
  depotGarantie: [null, [Validators.required, Validators.min(0)]]
})
```

Logique :

1. Au `ngOnInit` → `BienService.getMesBiens()` → GET `/users/biens`
2. Changement de `bienId` → auto-remplit `loyerMensuel`, `chargesMensuelles`, `depotGarantie` depuis le bien sélectionné
3. Soumission → `ContratService.create(dto)` → POST `/contrats`
4. Succès → navigue vers `/proprietaire/contrats`

8.2 ContratsListeComponent

Route : `/proprietaire/contrats` | `/locataire/contrats`

Signaux d'état :

```
contrats      = signal<ContratDTO[]>([])
loading       = signal<boolean>(false)
currentPage   = signal<number>(0)
totalPages    = signal<number>(0)
```



```
showSignModal    = signal<boolean>(false)
selectedContrat = signal<ContratDTO | null>(null)
signing          = signal<boolean>(false)
```

Filtres :

```
filterStatut      = signal<StatutContrat | ''>>('')
filterLoyerMin    = signal<number | null>(null)
filterLoyerMax    = signal<number | null>(null)
filterDateDebut   = signal<string>('')
filterDateFin     = signal<string>('')
pageSize         = 10
```

Logique :

1. Appelle `ContratService.search(dto)` → `POST /contrats/search`
2. **Modal de signature (canvas) :**
 - S'ouvre pour les contrats avec `statut === 'EN_ATTENTE_SIGNATURE_PROPRIO'`
 - Canvas HTML5 avec dessin libre souris + tactile
 - Bouton "Effacer" → `ctx.clearRect(0, 0, canvas.width, canvas.height)`
 - Soumission :

```
const base64 = canvas.toDataURL('image/png');
ContratService.signerProprietaire(id, { signatureBase64: base64 })
  → POST /contrats/:id/signer-proprio
```

- Après signature → recharge la liste
- 3. **Couleurs de statut :**
 - `BROUILLON` → gris
 - `EN_ATTENTE_SIGNATURE_PROPRIO` → orange
 - `EN_ATTENTE_SIGNATURE_LOCATAIRE` → jaune
 - `SIGNE` → vert
 - `EXPIRE` → rouge clair
 - `ANNULE` → rouge foncé

9. Module États des Lieux

9.1 EdlFormComponent

Route : `/proprietaire/etats-des-lieux/nouveau`

Signaux d'état :

```
biens          = signal<BienDTO[]>([])
loading        = signal<boolean>(false)
error          = signal<string | null>(null)
openPieces     = signal<Set<number>>(new Set()) // indices des sections expandées
```

Structure du formulaire (FormBuilder avec FormArrays) :

```
edlForm = fb.group({
  bienId: [null, Validators.required],
  emailLocataire: ['', [Validators.required, Validators.email]],
  type: ['ENTREE', Validators.required],
  dateRealisation: ['', Validators.required],
  heureRealisation: ['', ],
  observations: ['', ],
  compteurs: fb.array([], // FormArray de CompteurGroup
  cles: fb.array([], // FormArray de CleGroup
  pieces: fb.array([], // FormArray de PieceGroup
})
```

```
// Structure d'une pièce
PieceGroup = fb.group({
  nom:      ['', Validators.required],
  elements: fb.array([])           // FormArray de ElementGroup
})

// Structure d'un élément
ElementGroup = fb.group({
  type:      ['MUR', Validators.required],
  etat:      ['BON', Validators.required],
  commentaire: ['']
})

// Structure d'un compteur
CompteurGroup = fb.group({
  type:      ['EAU_FROIDE', Validators.required],
  valeur:    [0, Validators.required],
  unite:     ['m³', Validators.required]
})

// Structure d'une clé
CleGroup = fb.group({
  description: ['', Validators.required],
  quantite:    [1, [Validators.required, Validators.min(1)]]
})
```

Templates de pièces pré-définis :

```
const PIECE_TEMPLATES = [
  { nom: 'Entrée',      elements: ['MUR', 'PLAFOND', 'SOL', 'PORTE'] },
  { nom: 'Salon',       elements: ['MUR', 'PLAFOND', 'SOL', 'FENETRE', 'PRISE', 'LUMINAIRE'] },
  { nom: 'Chambre',     elements: ['MUR', 'PLAFOND', 'SOL', 'FENETRE', 'PORTE', 'PRISE', 'LUMINAIRE'] },
  { nom: 'Cuisine',     elements: ['MUR', 'PLAFOND', 'SOL', 'FENETRE', 'PRISE', 'LUMINAIRE', 'EQUIPEMENT'] },
  { nom: 'Salle de bain', elements: ['MUR', 'PLAFOND', 'SOL', 'FENETRE', 'PRISE', 'LUMINAIRE', 'RADIATEUR'] },
  { nom: 'WC',          elements: ['MUR', 'PLAFOND', 'SOL', 'PORTE'] },
  { nom: 'Couloir',     elements: ['MUR', 'PLAFOND', 'SOL'] }
];
```

Logique :

1. Au `ngOnInit` → `BienService.getMesBiens()` → GET `/users/biens`
2. Bouton "Ajouter pièce depuis template" → peuple le FormArray `pieces` avec éléments à `etat: 'BON'`
3. Sections expandables/repliables via `openPieces` (Set d'indices)
4. Soumission → `EdlService.create(dto)` → POST `/etats-des-lieux`
5. Succès → navigue vers `/proprietaire/etats-des-lieux`

9.2 EdlListeComponent

Route : `/proprietaire/etats-des-lieux` | `/locataire/etats-des-lieux`

Signaux d'état :

```
edls          = signal<EtatDesLieuxDTO[]>([])
loading       = signal<boolean>(false)
currentPage   = signal<number>(0)
totalPages    = signal<number>(0)
showSignModal = signal<boolean>(false)
selectedEdl   = signal<EtatDesLieuxDTO | null>(null)
signing       = signal<boolean>(false)
filterType    = signal<TypeEtat | ''>>('')
filterStatut  = signal<StatutEdl | ''>>('')
pageSize      = 10
```

Logique :

1. Appelle `EdlService.search(dto)` → `POST /etats-des-lieux/search`
2. **Indicateur "pire état"** : pour chaque EDL, détermine le pire `EtatElement` parmi tous les éléments
 - Ordre de gravité : `HORS_SERVICE` > `MAUVAIS` > `USAGE_NORMAL` > `BON`
3. **Modal de signature** (canvas, même logique que contrats) :
 - Pour `statut === 'EN_ATTENTE_SIGNATURE_PROPRIO'`
 - `EdlService.signerProprietaire(id, { signatureBase64 })` → `POST /etats-des-lieux/:id/signer-propretaire`
4. **Couleurs de statut** :
 - `BROUILLON` → gris
 - `EN_ATTENTE_SIGNATURE_PROPRIO` → orange
 - `EN_ATTENTE_SIGNATURE_LOCATAIRE` → jaune
 - `SIGNE` → vert
 - `EXPIRE` → rouge

10. Module Quittances

10.1 QuittanceFormComponent

Route : `/proprietaire/quittances/nouveau`

Signaux d'état :

```
biens          = signal<BienDTO[]>([])
loadingBiens   = signal<boolean>(false)
submitting     = signal<boolean>(false)
error          = signal<string | null>(null)
```

Formulaire :

```
quittanceForm = fb.group({
  bienId:      [null, Validators.required],
  emailLocataire: [' ', [Validators.required, Validators.email]],
  contratId:   [null],
  mois:        [currentMonth, [Validators.required, Validators.min(1), Validators.max(12)]],
  annee:        [currentYear, [Validators.required, Validators.min(2000)]],
  loyerMensuel: [null, [Validators.required, Validators.min(1)]],
  chargesMensuelles: [null, [Validators.required, Validators.min(0)]],
  dateEcheance: [' ', Validators.required]
})
// anneeOptions = [currentYear - 2, currentYear - 1, currentYear, currentYear + 1, currentYear + 2]
```

Logique :

1. Au `ngOnInit` → `BienService.getMesBiens()` → `GET /users/biens`
2. Changement de `bienId` → auto-remplit `loyerMensuel`, `chargesMensuelles` depuis le bien sélectionné
3. Soumission → `QuittanceService.create(dto)` → `POST /quittances`
4. Succès → navigue vers `/proprietaire/quittances`

10.2 QuittancesListeComponent

Route : `/proprietaire/quittances | /locataire/quittances`

Signaux d'état :

```

quittances      = signal<QuittanceDTO[]>([])
loading         = signal<boolean>(false)
currentPage     = signal<number>(0)
totalPages      = signal<number>(0)
showSignModal   = signal<boolean>(false)
selectedQuittance = signal<QuittanceDTO | null>(null)
signing         = signal<boolean>(false)
filterStatut    = signal<StatutQuittance | ''>>('')
filterAnnee     = signal<number | null>(null)
pageSize        = 10

```

Logique :

1. Appelle `QuittanceService.search(dto)` → `POST /quittances/search`
2. **Indicateur retard** : si `dateEcheance < aujourd'hui` ET `statut !== 'PAYEE'` → badge "En retard"
3. **Marquer comme payée** (propriétaire uniquement) :
 - Ouvre modal signature canvas
 - `QuittanceService.marquerPayee(id, { signatureBase64 })` → `POST /quittances/:id/marquer-payee`
4. **Couleurs de statut** :
 - `EN_ATTENTE` → bleu/gris
 - `PAYEE` → vert
 - `EN_RETARD` → orange
 - `IMPAYEE` → rouge

11. Dashboard

11.1 ProprietaireDashboardComponent

Route : `/proprietaire/dashboard`

Signaux d'état :

```

loading      = signal<boolean>(true)
biens        = signal<BienDTO[]>([])
contrats     = signal<ContratDTO[]>([])
quittances   = signal<QuittanceDTO[]>([])
edls         = signal<EtatDesLieuxDTO[]>([])
alertCount   = computed(() =>
  quittances().filter(q => q.statut === 'EN_RETARD' || q.statut === 'IMPAYEE').length
)

```

Logique — chargement parallèle via `forkJoin` :

```

ngOnInit(): void {
  forkJoin({
    biens:      BienService.getMesBiens(),
    contrats:   ContratService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' }),
    quittances: QuittanceService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' }),
    edls:       EdlService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' })
  }).subscribe(({ biens, contrats, quittances, edls }) => {
    this.biens.set(biens.slice(0, 3));      // 3 derniers biens
    this.contrats.set(contrats.contenu);    // 5 derniers contrats
    this.quittances.set(quittances.contenu); // 5 dernières quittances
    this.edls.set(edls.contenu);           // 5 derniers EDL
    this.loading.set(false);
  });
}

```

Affichage :

- Grille de 3 cartes biens (image, titre, loyer, ville)
- Tableau contrats : adresse, locataire, loyer, statut, date début
- Tableau quittances : mois/année, locataire, montant, statut + badge d'alerte rouge si `alertCount > 0`

- Tableau EDL : type, adresse, statut, date

11.2 LocataireDashboardComponent

Route : `/locataire/dashboard`

Signaux d'état :

```
loading    = signal<boolean>(true)
monBien    = signal<BienDTO | null>(null)
contrats   = signal<ContratDTO[]>([])
quittances = signal<QuittanceDTO[]>([])
edls       = signal<EtatDesLieuxDTO[]>([])
```

Logique — même `forkJoin` que propriétaire :

```
ngOnInit(): void {
  forkJoin({
    biens:      BienService.getMesBiens(),
    contrats:   ContratService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' }),
    quittances: QuittanceService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' }),
    edls:       EdlService.search({ page: 0, size: 5, sortBy: 'createdAt', sortDirection: 'DESC' })
  }).subscribe(({ biens, contrats, quittances, edls }) => {
    this.monBien.set(biens[0] ?? null); // Unique bien du locataire
    this.contrats.set(contrats.contenu);
    this.quittances.set(quittances.contenu);
    this.edls.set(edls.contenu);
    this.loading.set(false);
  });
}
```

12. Module Chat

12.1 ChatComponent

Route : `/proprietaire/chat | /locataire/chat`

Signaux d'état :

```
conversations = signal<ConversationDTO[]>([])
selectedConv  = signal<ConversationDTO | null>(null)
messages      = signal<ChatMessage[]>([])
loadingConvs  = signal<boolean>(false)
loadingMessages = signal<boolean>(false)
sending       = signal<boolean>(false)
messageInput  = signal<string>('')
searchNomBien = signal<string>('')
filterNonLu   = signal<boolean>(false)
convPage      = signal<number>(0)
convTotalPages = signal<number>(0)
```

Initialisation :

```
ngOnInit(): void {
  // 1. Charge les conversations
  this.loadConversations(0);

  // 2. Écoute les messages temps réel
  this.chatService.messages$.subscribe(msg => {
    if (this.selectedConv()?.bienId === msg.bienId) {
      // Déduplique puis ajoute
      if (!this.messages().some(m => m.id === msg.id)) {
        this.messages.update(list => [...list, buildChatMessage(msg, 'received')]);
      }
    }
  });
}
```

```

    }
  }
  this.loadConversations(0); // Rafraichit la liste pour mettre à jour lastMessage
});

// 3. Écoute les notifications
this.chatService.notifications$.subscribe(() => {
  this.chatService.unreadCount.update(n => n + 1);
});
}

```

Chargement des conversations :

```

loadConversations(page: number): void {
  const dto: ConversationSearchDTO = {
    nomDuBien:      this.searchNomBien() || undefined,
    lu:              this.filterNonLu() ? false : undefined,
    page,
    size:            12,
    sortBy:          'lastMessageAt',
    sortDirection:   'DESC'
  };
  ChatService.searchConversations(dto)
    → POST /conversations/search
}

```

Sélection d'une conversation :

```

selectConversation(conv: ConversationDTO): void {
  this.selectedConv.set(conv);
  const emailInterlocuteur = conv.emailExpediteur === currentUserEmail
    ? conv.emailDestinataire
    : conv.emailExpediteur;

  // Charge l'historique
  ChatService.getHistorique(conv.bienId, emailInterlocuteur)
    → GET /historique?bienId={bienId}&emailInterlocuteur={email}
  // Transforme en ChatMessage[] avec séparateurs de dates

  // Marque comme lu
  ChatService.marquerLus(emailInterlocuteur)
    → POST /messages/conversation/{email}/lire
}

```

Transformation MessageDTO[] → ChatMessage[] (avec séparateurs de date) :

```

function buildChatMessages(msgs: MessageDTO[], currentUserEmail: string): ChatMessage[] {
  const result: ChatMessage[] = [];
  let lastDate: string | null = null;

  msgs.forEach(msg => {
    const dateStr = formatDate(msg.createdAt); // ex: "17 mai 2026"
    if (dateStr !== lastDate) {
      result.push({ type: 'separator', label: dateStr });
      lastDate = dateStr;
    }
    result.push({
      type:      'message',
      id:        msg.id,
      contenu:   msg.contenu,
      direction: msg.expediteurEmail === currentUserEmail ? 'sent' : 'received',
      time:      formatTime(msg.createdAt),
      isNew:     !msg.lu
    });
  });
}

```

```
    return result;
  }
}
```

Envoi d'un message :

```
sendMessage(): void {
  if (!this.messageInput().trim() || !this.selectedConv()) return;
  const conv = this.selectedConv();
  const dest = conv.emailExpéditeur === currentUserEmail
    ? conv.emailDestinataire
    : conv.emailExpéditeur;

  // Envoi via WebSocket
  ChatService.send(this.messageInput(), dest, conv.bienId);
  // → PUBLISH /app/chat.send { contenu, emailDestinataire, bienId }

  // Ajout optimiste dans la liste locale
  this.messages.update(list => [...list, {
    type: 'message',
    id: Date.now(),
    contenu: this.messageInput(),
    direction: 'sent',
    time: formatTime(new Date().toISOString())
  }]);
  this.messageInput.set('');
}
```

Raccourcis clavier :

- **Enter** → envoie le message
- **Shift+Enter** → insère une nouvelle ligne

13. Pages Publiques de Signature

Ces pages sont accessibles **sans authentification**. Elles sont utilisées quand le back-end envoie un lien par email au signataire (locataire ou propriétaire).

13.1 SignatureContratComponent

Route : `/contrats/signer/:token`

Signaux d'état :

```
loading = signal<boolean>(true)
contrat = signal<ContratDTO | null>(null)
expired = signal<boolean>(false)
notFound = signal<boolean>(false)
signing = signal<boolean>(false)
success = signal<boolean>(false)
error = signal<string | null>(null)
```

Logique :

```
ngOnInit(): void {
  const token = this.route.snapshot.paramMap.get('token')!;
  ContratService.getByToken(token)
    → GET /contrats/signer/:token (sans JWT)
    .subscribe({
      next: (contrat) => {
        this.contrat.set(contrat);
        this.loading.set(false);
      },
      error: (err) => {
        if (err.status === 410) this.expired.set(true);
        if (err.status === 404) this.notFound.set(true);
      }
    });
}
```

```

        this.loading.set(false);
    }
});
}

signer(): void {
    // 1. Vérifie que le canvas n'est pas vide
    if (this.isCanvasEmpty()) {
        this.error.set('Veuillez signer dans le cadre prévu');
        return;
    }
    // 2. Récupère l'image en base64
    const base64 = this.canvasRef.nativeElement.toDataURL('image/png');
    // 3. Envoie la signature
    this.signing.set(true);
    ContratService.signerParToken(token, { signatureBase64: base64 })
        → POST /contrats/signer/:token (sans JWT)
        .subscribe({
            next: () => this.success.set(true),
            error: (err) => this.error.set(err.userMessage)
        });
}

```

Gestion du canvas (souris + tactile) :

```

// Événements souris
canvas.addEventListener('mousedown', (e) => { isDrawing = true; ctx.beginPath(); ctx.moveTo(x, y); });
canvas.addEventListener('mousemove', (e) => { if (isDrawing) ctx.lineTo(x, y); ctx.stroke(); });
canvas.addEventListener('mouseup', () => { isDrawing = false; });

// Événements tactile (mobile)
canvas.addEventListener('touchstart', (e) => { e.preventDefault(); /* même logique */ });
canvas.addEventListener('touchmove', (e) => { e.preventDefault(); /* même logique */ });
canvas.addEventListener('touchend', () => { isDrawing = false; });

// Vérification canvas vide
isCanvasEmpty(): boolean {
    const data = ctx.getImageData(0, 0, canvas.width, canvas.height).data;
    return !data.some(pixel => pixel !== 0);
}

// Effacer
clearCanvas(): void {
    ctx.clearRect(0, 0, canvas.width, canvas.height);
}

```

États d'affichage :

- loading === true → spinner / skeleton
- expired === true → message "Ce lien a expiré ou n'est plus valide"
- notFound === true → message "Lien de signature introuvable"
- success === true → message de confirmation de signature
- Sinon → affiche les infos du contrat + canvas de signature

Informations du contrat affichées :

- Adresse du bien
- Nom propriétaire / locataire
- Loyer mensuel + charges + dépôt de garantie
- Date début + durée bail
- Statut actuel

13.2 SignatureEdlComponent

Route : /edl/signer/:token

Signaux d'état :


```
loading = signal<boolean>(true)
edl      = signal<EtatDesLieuxDTO | null>(null)
expired  = signal<boolean>(false)
notFound = signal<boolean>(false)
signing  = signal<boolean>(false)
success  = signal<boolean>(false)
error    = signal<string | null>(null)
```

Logique :

```
ngOnInit(): void {
  const token = this.route.snapshot.paramMap.get('token')!;
  EdlService.getByToken(token)
    → GET /etats-des-lieux/signer/:token (sans JWT)
    .subscribe({
      next: (edl) => {
        this.edl.set(edl);
        this.loading.set(false);
      },
      error: (err) => {
        if (err.status === 401 || err.status === 410) this.expired.set(true);
        if (err.status === 404) this.notFound.set(true);
        this.loading.set(false);
      }
    });
}

signer(): void {
  if (this.isCanvasEmpty()) {
    this.error.set('Veuillez apposer votre signature');
    return;
  }
  const base64 = this.canvasRef.nativeElement.toDataURL('image/png');
  this.signing.set(true);
  EdlService.signerParToken(token, { signatureBase64: base64 })
    → POST /etats-des-lieux/signer/:token (sans JWT)
    .subscribe({
      next: () => this.success.set(true),
      error: (err) => this.error.set(err.userMessage)
    });
}
```

Informations de l'EDL affichées :

- Type (ENTREE / SORTIE) + date + heure
- Adresse du bien + type de bien
- Nom propriétaire / locataire
- **Tableau des compteurs** : type, valeur, unité
- **Tableau des clés** : description, quantité
- **Pièces (accordéon)** : nom de la pièce → liste des éléments avec type, état, commentaire
- Observations générales
- Canvas de signature (même logique souris + tactile que `SignatureContratComponent`)

Récapitulatif de tous les appels API

Méthode	URL	JWT	Corps	Retour	Composant
POST	/auth/login	Non	{email, password}	{accessToken}	LoginComponent
POST	/auth/register	Non	FormData	{accessToken}	RegisterComponent
POST	/auth/logout	Oui	{}	void	AuthService
POST	/auth/refresh	Non	{}	{accessToken}	JwtInterceptor
GET	/auth/me	Oui	—	UserDTO	AuthService

Méthode	URL	JWT	Corps	Retour	Composant
POST	/auth/forgot-password?email=	Non	null	string	ForgotPasswordComponent
POST	/auth/reset-password?token=&newPassword=	Non	null	string	ResetPasswordComponent
POST	/biens/search	Non	BienSearchDTO	BienPageDTO	BiensListeComponent
GET	/biens/:id	Non	—	BienDTO	BienDetailComponent
POST	/biens	Oui	FormData	void	BienFormComponent
GET	/users/biens	Oui	—	BienDTO[]	MesBiensComponent, MonLogementComponent, Dashboard
POST	/biens/:bienId/assigne-locataire/:userId	Oui	{}	void	MesBiensComponent
POST	/users/:bienId/recherche-locataire	Oui	LocataireSearchDTO	LocatairePageDTO	MesBiensComponent
POST	/contrats	Oui	ContratFormDTO	void	ContratFormComponent
POST	/contrats/search	Oui	ContratSearchDTO	ContratPageDTO	ContratsListeComponent, Dashboard
POST	/contrats/:id/signer-proprio	Oui	SignatureDTO	void	ContratsListeComponent
GET	/contrats/signer/:token	Non	—	ContratDTO	SignatureContratComponent
POST	/contrats/signer/:token	Non	SignatureDTO	void	SignatureContratComponent
POST	/etats-des-lieux	Oui	EtatDesLieuxFormDTO	void	EdlFormComponent
POST	/etats-des-lieux/search	Oui	EdlSearchDTO	EdlPageDTO	EdlListeComponent, Dashboard
POST	/etats-des-lieux/:id/signer-proprietaire	Oui	SignatureDTO	void	EdlListeComponent
GET	/etats-des-lieux/signer/:token	Non	—	EtatDesLieuxDTO	SignatureEdlComponent
POST	/etats-des-lieux/signer/:token	Non	SignatureDTO	void	SignatureEdlComponent
POST	/quittances	Oui	QuittanceFormDTO	void	QuittanceFormComponent
POST	/quittances/search	Oui	QuittanceSearchDTO	QuittancePageDTO	QuittancesListeComponent, Dashboard
POST	/quittances/:id/marquer-payee	Oui	SignatureDTO	void	QuittancesListeComponent
GET	/quittances/bien/:bienId	Oui	—	QuittanceDTO[]	QuittancesListeComponent
GET	/quittances/mes-quittances	Oui	—	QuittanceDTO[]	QuittancesListeComponent (locataire)
GET	/quittances/:id	Oui	—	QuittanceDTO	QuittancesListeComponent
POST	/conversations/search	Oui	ConversationSearchDTO	ConversationPageDTO	ChatComponent
GET	/historique?bienId=&emailInterlocuteur=	Oui	—	MessageDTO[]	ChatComponent, BienDetailComponent
POST	/messages/conversation/:email/lire	Oui	{}	void	ChatComponent
GET	/messages/non-lus	Oui	—	number	ChatService (init)
WS PUBLISH	/app/chat.send	Via WS	{contenu, emailDestinataire, bienId}	—	ChatComponent, BienDetailComponent
WS SUB	/user/{email}/queue/messages	Via WS	—	MessageDTO	ChatService
WS SUB	/user/{email}/queue/notifications	Via WS	—	NotificationDTO	ChatService

Notes importantes pour le développeur

1. **Structure de pagination universelle** — Toutes les réponses paginées suivent le même modèle : `{ contenu[], pageActuelle, totalPages, totalElements, dernierePage, premierePage }`
2. **Signature canvas** — Même logique pour contrats, EDL et quittances :
 - Canvas HTML5 → dessin souris + tactile
 - `canvas.toDataURL('image/png')` → base64
 - Envoyé dans `{ signatureBase64: string }`
3. **WebSocket** — Dépendances npm requises :

```
npm install @stomp/stompjs sockjs-client
npm install --save-dev @types/sockjs-client
```
4. **Déduplication messages** — Toujours vérifier `!messages().some(m => m.id === incoming.id)` avant d'ajouter un message reçu via WebSocket pour éviter les doublons avec l'historique HTTP.
5. **Routes publiques de signature** — Le `JwtInterceptor` NE doit PAS injecter de token sur :
 - `GET/POST /contrats/signer/:token`
 - `GET/POST /etats-des-lieux/signer/:token`
6. **Indicateur pire état EDL** — Ordre de priorité : `HORS_SERVICE` (4) > `MAUVAIS` (3) > `USAGE_NORMAL` (2) > `BON` (1)
7. **Auto-population des formulaires** — `ContratForm` et `QuittanceForm` récupèrent `loyerMensuel` et `chargesMensuelles` directement depuis le `BienDTO` sélectionné, sans appel API supplémentaire.